FINANCIAL RISK PLATFORM

Architecture & System Design

Working Draft v0.1 | 15 October 2026

Proposed target architecture, component responsibilities, runtime flows, API boundaries, security controls, deployment topology and release gates.

1. Purpose and status

This document defines the proposed target architecture and system design for a configurable financial-risk platform intended to replace selected Moody's calculation and balance-sheet risk capabilities. It complements the Detailed Data Model and the synchronized frontend/backend workstream plans.

Status: working design baseline, not a production-approved architecture. Client infrastructure, exact Moody's modules in scope, source feed contracts, regulatory jurisdictions, non-functional targets, hosting constraints and security policies require confirmation.

> MVP principle: broad platform architecture, deliberately bounded initial implementation. Build durable control points and extension contracts; limit initial volumes, integrations, instrument coverage and calculation depth until validated requirements and benchmark data are available.

2. Design objectives and principles

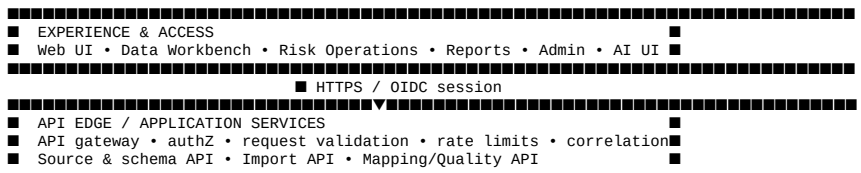
- Replace the calculation capability that the client actually uses, not reproduce proprietary internals or assume every Moody's module is licensed.
- Separate source payloads, canonical financial data, configurable rules and deterministic calculation services.
- Treat financial data quality, lineage, security, auditability and controlled change as first-class platform capabilities.
- Accept uncertain source shapes through registered contracts, profiling and versioned mappings. Never silently drop or reinterpret unknown fields.
- Make each calculation reproducible from pinned input snapshots and exact versions of schema, mappings, instruments, rules, assumptions and engine.
- Keep AI outside the authoritative calculation path. AI may explain, retrieve, propose or orchestrate approved actions, with permissions and human approval.
- Use modular boundaries and versioned APIs so ALM/IRRBB, collateral and capital modules can be added without rewriting ingestion or orchestration.

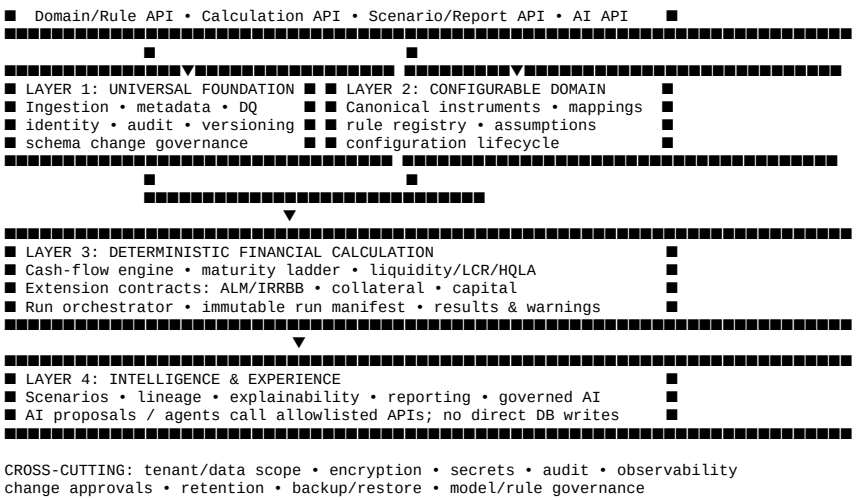
3. Scope and assumptions

Area	Proposed baseline	Confirmation required
Initial vertical slice	Configurable ingestion → mapping → quality checks → canonical records → cash flows/maturity ladder → one client-approved liquidity/LCR calculation → reconciliation/report	Formula, jurisdiction, benchmark cases, tolerances
Source inputs	CSV/XLSX/JSON samples and provisional synthetic contracts; adapter interfaces for warehouse/API/batch sources	Actual warehouse schemas, ownership, cadence, transport
Financial modules	Liquidity/LCR/HQLA and supporting cash-flow/maturity functions first; ALM/IRRBB, collateral and capital as extension points	Licensed modules and business priority
Deployment	Environment-neutral reference design; containerized services are a proposed default	Cloud/on-prem, network, database, IAM, key management
AI	Optional, governed assistance; no dependency for core calculations	Approved model/provider, data residency, retention
Scale	Small initial datasets with production-minded interfaces and controls	Expected volumes, SLAs, batch windows, concurrency

Do not treat any example formula, threshold, product attribute or regulatory interpretation in this design as approved policy. Financial experts must approve the rule definitions and golden test cases.

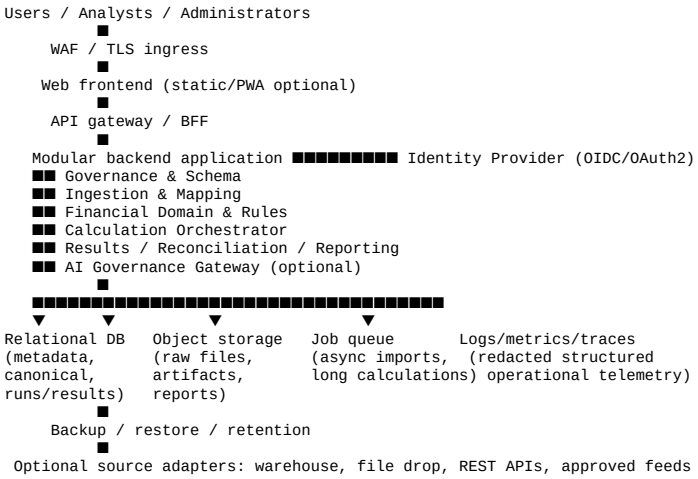
4. Logical architecture





The four layers are logical boundaries, not a mandate to deploy four separate applications. Start as a modular application with clear service/module boundaries; split into independently deployed services only where scaling, isolation, ownership or operational needs justify it.

5. Recommended deployment topology



- Reference implementation: React/TypeScript frontend and a typed backend API (FastAPI is a reasonable Python-oriented option for data and calculation workloads); final framework must match the team and client environment.
- Use a relational database for configuration, identity references, metadata, canonical entities, run manifests and results. Use object storage for source files, large artifacts and generated reports.
- Use an asynchronous job mechanism for imports, profiling, large calculations and report generation. A lightweight database-backed job queue may be sufficient for the MVP; preserve an adapter boundary for a dedicated broker later.
- Keep the deployment cloud-neutral until client hosting and security requirements are confirmed. Avoid committing to a specific vendor from this draft alone.

6. Core runtime flows

6.1 Ingestion and schema-change flow

1. Register source system/profile and a provisional or approved source contract.
2. Upload or receive a file/feed; create an import job and immutable artifact reference, hash, timestamps and uploader/source identity.
3. Profile columns, types, null rates, distinct values and value distributions. Unknown fields are retained in the raw artifact and surfaced as candidates.
4. Apply an explicit, versioned source-to-canonical mapping. Mapping validation checks required fields, conversions, units, date semantics and key uniqueness.
5. Run data-quality rules. Accepted records enter a versioned canonical dataset snapshot; invalid records are rejected or quarantined with actionable issue codes.
6. Run source-to-target reconciliation where totals/counts are meaningful. Record all warnings and approvals.
7. If drift is detected, create a review/change proposal. Do not auto-promote schema, mapping or rule changes into production.

6.2 Calculation and result flow

1. User selects dataset snapshot, calculation module/version, reporting date, rule-set version, assumption set and scenario.
2. API authorizes access and validates that required inputs and approved configuration versions exist.
3. Orchestrator creates a run record and immutable run manifest; the job moves through queued, running, validation, completed/failed states.
4. Calculation service executes deterministic, unit-tested logic against a stable input snapshot. It records intermediate values, warnings and lineage references.
5. Post-run checks validate invariants, totals, expected row counts and reconciliation tolerances. A failed control marks the run as failed or requiring review, never silently successful.
6. Persist results and manifest; present metrics, warnings, source-to-result lineage and comparison to benchmark/baseline.
7. Any rerun after changing input or configuration creates a new run. Do not overwrite historic results.

6.3 Governed AI workflow

1. User asks for an explanation, impact assessment or a proposed configuration change.
2. AI gateway checks identity, scope, data classification, approved model route and permitted tools.
3. Retrieval is limited to authorized records/documents; references are stored with the AI request.
4. AI returns a grounded explanation or a structured proposal, with uncertainty and evidence links.
5. Proposal is validated by deterministic services and routed for human approval where consequential.
6. Only approved actions are executed through allowlisted APIs. Tool calls, inputs/outputs metadata, approvals and evaluation results are audited.

7. Component responsibilities

Component	Responsibilities	Must not do
API edge / BFF	Authentication handoff, request validation, rate limits, correlation IDs, consistent errors	Contain business formulas or bypass service authorization
Schema & contract registry	Schema versions, fields, source profiles, change proposals, compatibility checks	Silently mutate approved contracts
Ingestion adapters	Parse source formats, preserve raw artifact, emit normalized records and provenance	Assume all sources share one shape
Mapping & quality	Versioned transformations, type/unit conversions, validation issues, quarantine, profiling and drift	Drop unknown fields without trace
Canonical domain	Instrument, position, counterparty, contractual terms, market/reference data and stable identifiers	Make client-specific assumptions invisible
Rule/configuration registry	Versioned rules, assumptions, effective dates, approval state and module compatibility	Edit a version already used by completed runs
Calculation modules	Pure/deterministic financial calculations where possible; intermediate outputs and warnings	Call LLMs to produce authoritative numbers
Run orchestrator	Pin versions, schedule jobs, retries, idempotency, state transitions and lineage manifest	Allow duplicate side effects on retry
Reconciliation & reporting	Compare source, canonical and calculated totals; reports with run/version context	Hide material breaks or warnings
AI governance gateway	Authorized retrieval, tool allowlist, proposal/evaluation/audit	Write directly to database or approve its own changes
Audit/observability	Immutable audit trail, metrics, traces, alerts and redaction	Log secrets or unnecessary raw financial data

8. Canonical financial domain and data boundaries

• Raw zone: original source artifacts and immutable metadata. Restrict access; apply retention and encryption policies.

- Staging zone: parsed source rows with source record identifiers, import batch, raw-field map and parsing status.
- Canonical zone: normalized entities with internal stable IDs, source identifiers, validity/effective dates, version and provenance.
- Configuration zone: schema/mapping/rule/assumption/module versions and approval history.
- Calculation zone: run manifests, input references, intermediate values, result records, warnings and lineage edges.
- Presentation zone: report instances and views derived from authorized results; reports carry run ID and configuration versions.
- AI evidence zone: request metadata, retrieval references, tool-call metadata, proposals, approvals and evaluation outcomes. Avoid duplicating raw sensitive content unless explicitly approved.

For financial values, use fixed-precision decimal types and explicit currency/unit fields. Preserve source precision and define rounding points per approved calculation specification. Store dates, timestamps, time zones, business dates and effective dates distinctly.

9. API design and contracts

Proposed REST contract prefix: /api/v1. Endpoint names below are illustrative and must be reconciled with the synchronized workstream plans and finalized in OpenAPI before implementation.

Domain	Representative endpoints	Notes
Identity/context	GET /api/v1/me; GET /api/v1/me/permissions	Identity provider is authoritative; server enforces every request
Schemas	GET /api/v1/schemas; GET /api/v1/schemas/{id}/versions; POST /api/v1/schema-change-proposals	Immutable published versions; changes reviewed
Imports	POST /api/v1/datasets/imports; GET /api/v1/imports/{id}; POST /api/v1/imports/{id}/validate	Large file upload may use pre-signed object storage URL if approved
Mappings	GET /api/v1/mappings; POST /api/v1/imports/{id}/mappings/validate; POST /api/v1/mappings/{id}/versions	Versioned mappings and explicit promotion
Quality	GET /api/v1/imports/{id}/issues; GET /api/v1/imports/{id}/profile; POST /api/v1/reconciliations	Paginated issue details; raw values masked by policy
Rules/modules	GET /api/v1/rules; GET /api/v1/calculation-modules; POST /api/v1/rule-proposals	Rule promotion requires validation/approval
Runs/results	POST /api/v1/calculation-runs; GET /api/v1/calculation-runs/{id}; GET /api/v1/calculation-runs/{id}/results	Run creation idempotency key; async status and result references
Lineage	GET /api/v1/calculation-runs/{id}/lineage	Source-to-result provenance
Scenarios	POST /api/v1/scenarios; POST /api/v1/scenarios/{id}/run; GET /api/v1/scenarios/{id}	Scenario is versioned and references baseline
Audit/reports	GET /api/v1/audit-events; POST /api/v1/reports; GET /api/v1/reports/{id}	Permission-filtered; audit access itself is audited
AI	POST /api/v1/ai/requests; GET /api/v1/ai/requests/{id}; POST /api/v1/ai/proposals/{id}/approve	AI endpoints can be disabled without disabling core calculations

- Use a consistent error envelope: code, message, details, correlation_id and retryable.
- Use pagination, filtering, request size limits, timeouts and schema validation.
- Use idempotency keys for run/import creation and safe retry semantics for asynchronous jobs.
- Expose OpenAPI as the source of truth for API contracts; contract tests must run in CI.
- Never rely on frontend-only checks for authorization or validation.

10. Security, privacy and governance

- Authentication: enterprise identity provider using OIDC/OAuth2 where available; do not invent a parallel identity store unless required.
- Authorization: RBAC with data-scope constraints; enforce tenant/entity/dataset permissions on the server and at query boundaries.
- Least privilege: separate roles for platform admin, data steward, risk analyst, approver, auditor and service identity; final role model requires client approval.
- Encryption: TLS in transit; database, object storage, backups and secrets encrypted at rest using approved key management.
- Secrets: managed secret store or equivalent; no credentials in source control, frontend bundles, logs or documents.
- Audit: record actor/service, action, object, timestamp, correlation ID, outcome, approval reference and version identifiers. Protect audit records against ordinary modification.
- Data minimization: mask or redact raw values in logs and UI where unnecessary; restrict raw artifacts and quarantined records.
- Change governance: maker-checker approval for production schema, mapping, rule, assumption and module configuration changes where required.
- AI controls: approved model routing, prompt/data classification, retrieval access checks, tool allowlist, prompt-injection defenses, output validation, human approval and retention controls.
- Operational security: dependency scanning, static analysis, secret scanning, vulnerability remediation, penetration testing and incident response.

No jurisdiction-specific compliance claim is made here. Data residency, retention, privacy, encryption key ownership, segregation-of-duties and regulatory evidence requirements must be confirmed with the client.

11. Reliability, performance and operations

Concern	MVP design	Production gate
Async work	Durable job state, retry count, failure reason, dead-letter/review state	Load tests, queue recovery, concurrency limits
Idempotency	Idempotency key for run/import creation; duplicate detection	Prove retries cannot double-publish or corrupt results
Transactions	Atomic state changes for config promotion and run manifest creation	Concurrency and isolation review
Observability	Structured logs, metrics, traces, correlation IDs; redact sensitive payloads	Dashboards, alerts, SLOs and on-call runbooks
Backup/restore	Automated DB/object backups and documented restore procedure	Demonstrated restore and recovery objectives
Performance	Bounded datasets; measure import, validation and calculation time	Client-approved volumes, batch windows and p95/p99 targets
Availability	Graceful failure and resumable jobs where practical	HA, disaster recovery and failover exercises
Retention	Configurable retention policy by artifact/result class	Legal/business approval and deletion evidence

Do not invent numeric SLAs before discovery. Record the target, test dataset size, environment and measurement method when the client agrees service-level objectives.

12. Calculation correctness and reproducibility

- Every run has a unique ID and immutable manifest including reporting date, source dataset snapshot, schema version, mapping version, instrument model/version, rule-set version, assumptions, scenario, module version, engine build, calculation parameters and user/service identity.
- Store explicit references to input records and source artifacts; maintain lineage from result to intermediate value to canonical input to source record.
- Use deterministic calculation code with fixed-precision arithmetic, explicit rounding rules, stable ordering where relevant and controlled random seeds for any approved stochastic method.
- Maintain golden datasets and independently approved expected outputs for each financial module and supported instrument type.
- Run unit tests, property/invariant tests, regression tests, reconciliation tests and parallel comparisons against the incumbent system where permitted.
- Define tolerances per metric and explain whether a comparison is exact, rounded, or tolerance-based.

- Never label a module production-ready until domain experts approve formulas, coverage, edge cases, regulatory interpretation and reconciliation evidence.

13. AI and agentic design guardrails

- AI is an optional application layer, not a dependency of the deterministic financial calculation path.
- Use retrieval augmented with authorized source documents, rule definitions, model documentation and run evidence; cite the evidence in explanations.
- Agent tools are explicit allowlisted APIs with typed inputs, scoped permissions, timeouts and auditable outputs.
- AI-generated mappings, rules or assumptions are proposals. They require validation and approval before promotion.
- Do not let an agent execute arbitrary SQL, modify historical run results, change permissions, or approve its own proposal.
- Evaluate AI for groundedness, correctness, leakage, refusal/permission behavior, tool safety and regression; store evaluation version and outcomes.
- External model APIs are optional and must be approved for data residency, contractual terms, privacy and retention. The core product must function when AI is disabled.

14. Frontend system design

Workspace	Primary screens	Core interaction
Platform administration	Users/roles, access scopes, audit, environment settings	Manage access and inspect audited actions
Source & schema registry	Sources, contracts, schema versions, drift/change proposals	Review unknown fields and promote approved versions
Ingestion workbench	Upload/import status, mapping grid, profile, validation issues, quarantine	Map fields, test conversions, correct/retry rejected records
Financial domain	Instrument types, instrument records, assumptions, rule registry	Inspect effective versions and approval status
Calculation operations	Module selection, reporting date, dataset/version selection, run history	Submit run, monitor status, inspect warnings and results
Reconciliation & scenarios	Baseline/scenario comparison, break summary, lineage drilldown	Investigate deltas and record review outcomes
Reports	Run-linked reports, exports, filters	Generate and retrieve authorized artifacts
AI assistant (optional)	Evidence-backed explanations, impact analysis, proposed changes	Review citations, accept/reject proposals, see tool-call audit

UI state must reflect backend lifecycle states. Do not show a calculation as successful until the backend run state and post-run controls confirm completion. Every result view should display the run ID, reporting date and configuration version context.

15. Proposed repository and code organization

```
/frontend
  /src
    /app              routes, layouts, auth/session
    /features         schemas, imports, mappings, domain, runs, reports, AI
    /components       shared UI components
    /api              generated/typed API client
    /types            shared API types
/backend
  /app
    /api/v1           route handlers and request/response schemas
    /core              config, authZ, errors, correlation, logging
    /governance        schema registry, approvals, audit
    /ingestion         parsers, source adapters, import jobs
    /mapping           transformations, validation, profiling
    /domain            instruments, positions, cashflows, reference data
    /rules             rule registry, assumptions, promotion
    /calculations      engine interface and module implementations
    /orchestration     run lifecycle, idempotency, job dispatch
    /reconciliation    comparisons, tolerances, break management
    /lineage            lineage graph and query services
    /reporting          report definitions and renderers
    /ai                 retrieval, tools, proposals, evaluation
    /db                 models, repositories, migrations
    /tests              unit, contract, integration, regression
/infra
  /local              local development composition
  /deployment         environment templates and manifests
```

```
/docs
  architecture, ADRs, OpenAPI, data dictionary, runbooks, threat model
```

Treat this as a suggested structure, not a requirement to create a large microservice repository on day one. Keep domain modules independently testable and dependency direction explicit.

16. Delivery phases and workstream synchronization

Phase	Primary outcome	Workstreams
P0	Confirm assumptions, source samples, security classifications, first calculation slice, golden-test plan	All; WS1 coordinates
P1	Identity/access, audit, schema registry, versioning, API skeleton and observability baseline	WS1; paired FE/BE contracts
P2	Import artifacts, source profiles, mappings, validation, quarantine, profiling and drift review	WS2
P3	Canonical instrument/domain model, rule/assumption registry, module interface and cash-flow foundations	WS3
P4	Run orchestration, first bounded liquidity calculation, reconciliation and result manifest	WS3 + WS4
P5	Scenarios, lineage UI, reporting, governed AI proposals and evaluation	WS4 + WS5
P6	Regression, security, performance, restore tests, documentation and second-module extension proof	All

- Maintain five synchronized pairs: WS1 Data Governance, Security & Platform Foundation; WS2 Data Ingestion, Mapping & Data Quality; WS3 Financial Domain Model & Calculation Services; WS4 Risk Operations, Scenarios, Reconciliation & Reporting; WS5 AI/GenAI, Automation & Innovation.
- Feature IDs, phase IDs, endpoint contracts, state names and acceptance criteria must stay aligned between each frontend and backend plan.
- Use a contract-first workflow: agree API/schema → update OpenAPI and data model → implement backend → implement frontend against contract → run contract/integration/regression tests.
- Changes to canonical schema, API contract, calculation rules or lifecycle states require an impact assessment across paired workstreams.

17. Testing strategy and release gates

- Unit: parser edge cases, mapping conversions, financial functions, rounding, rule evaluation and state transitions.
- Contract: OpenAPI request/response schemas, error envelopes, pagination and idempotency.
- Integration: source artifact through canonical data, calculation, result persistence, lineage and report.
- Security: authorization matrix, data-scope isolation, permission revocation, audit integrity, upload limits and injection defenses.
- Data quality: nulls, duplicates, invalid dates, unit/currency mismatch, missing references, outliers and unknown columns.
- Financial regression: approved golden cases, boundary cases, negative/zero values, maturity cutoffs, day-count conventions and supported instrument types.
- Reconciliation: source-to-canonical counts/totals and incumbent-vs-new results with metric-specific tolerances.
- AI evaluation: grounded citations, unauthorized retrieval attempts, unsafe tool requests, proposal validation and regression.
- Operational: retry, restart, partial failure, backup/restore, load and concurrency tests.

Release gates: no production calculation sign-off without named financial owner, approved calculation specification, golden dataset, tolerance policy, defect disposition, lineage evidence and documented rollback.

18. Architecture decisions to resolve

Decision	Current recommendation	Owner / evidence needed
Hosting topology	Cloud-neutral reference; containerized modular backend	Client infrastructure/security team
Backend framework	Typed REST backend; FastAPI is a viable candidate	Engineering team and existing platform constraints
Database	Relational store for metadata/configuration/canonical entities/results	DBA, volume, reporting and HA needs
Object storage	Use for immutable source artifacts and reports	Client storage/security policy
Job execution	Async workers; start lightweight, retain broker adapter boundary	Expected workload and batch windows
Identity provider	Integrate with enterprise OIDC/OAuth2 where available	IAM team
Source integration	Start with file-based adapter, then warehouse/API adapters after discovery	Data owners and actual feed samples
Regulatory scope	Confirm jurisdiction, reporting definitions and licensed module inventory	Risk/regulatory product owner
AI provider	Optional and approval-gated; no core dependency	Security, legal, procurement, data residency
Retention and RPO/RTO	Do not set numeric values until agreed	Business continuity and compliance owners

19. Risks and mitigations

Risk	Mitigation
Unknown or changing source schemas	Raw preservation, profiling, versioned contracts/mappings, drift review and quarantine
Incorrect financial outputs	Domain owner, approved formulas, golden datasets, parallel runs and explicit tolerances
Untraceable results	Immutable run manifest, record-level lineage and version-pinned configuration
Scope expansion to all modules at once	Platform-wide architecture; staged vertical slices and explicit module readiness gates
AI fabricates or changes financial logic	Deterministic calculation authority; proposal-only AI; allowlisted tools and approval
Sensitive data leaks through logs/AI	Classification, redaction, access controls, approved model route and retention controls
Frontend/backend drift	Shared feature IDs, OpenAPI contract-first workflow and CI contract tests
Overengineering early	Modular monolith first, deployable boundaries later, measured scaling decisions

20. Definition of done for the first end-to-end slice

- A user with the correct role can register a source, import a sample dataset and inspect a data profile.
- Unknown columns are retained in the source artifact and visibly flagged for review.
- A versioned mapping can be validated and used to create canonical records; invalid rows are quarantined with clear issue codes.
- Approved configuration versions are pinned to a calculation run manifest.
- The chosen calculation slice executes deterministically and emits outputs, intermediate values, warnings and lineage.
- Reconciliation checks and approved golden tests pass within agreed tolerances.
- Results are searchable by run ID and show reporting date, dataset snapshot, rules, assumptions, engine version and status.
- All privileged actions and configuration changes are audited; server-side access checks are tested.
- AI features can be disabled without breaking ingestion or financial calculations.

- Frontend/backend paired contracts and test evidence are published alongside the release.

Appendix A. Key entities and relationship summary

- SourceSystem → SourceProfile → SourceContract → SchemaVersion / FieldMappingVersion.
- ImportJob → ImportArtifact → ImportBatch → SourceRecord → CanonicalRecord; ValidationIssue and QuarantineItem reference relevant records/batches.
- CanonicalRecord links to domain entities such as Instrument, Position, Counterparty, ContractualTerm, CashflowEvent, CollateralAsset and MarketObservation.
- RuleSetVersion + AssumptionSet + CalculationModuleVersion + DatasetSnapshot + ScenarioVersion → CalculationRun.
- CalculationRun → CalculationInputRef, IntermediateValue, CalculationWarning, domain result tables, ReconciliationRun and ReportInstance.
- CalculationRun → LineageNode/LineageEdge → source/canonical inputs and output metrics.
- AIRequest → RetrievalReference / ToolCall / Proposal / EvaluationRun; approved proposals create ordinary governed change records.

Appendix B. Standard lifecycle states (proposed)

Object	Candidate states
Import job	created, uploaded, profiling, mapping_required, validating, quarantined, ready, failed, cancelled
Schema/mapping/rule version	draft, in_review, approved, active, deprecated, rejected
Calculation run	queued, validating, running, post_check, completed, completed_with_warnings, failed, cancelled
Change proposal	draft, submitted, in_review, approved, rejected, applied, superseded
Review task	open, assigned, approved, rejected, deferred, closed
Report instance	queued, generating, completed, failed, expired

Use one canonical spelling and transition table per object in implementation. Every transition must define authorized actor, preconditions, side effects and audit event.

Appendix C. Glossary

Term	Meaning
Canonical model	Normalized internal representation of financial entities and attributes
Source contract	Versioned description of expected source shape and semantics
Golden dataset	Expert-approved input set with expected outputs used for regression
Run manifest	Immutable list of data/configuration/code versions and parameters for one calculation
Lineage	Trace from result to intermediate calculation and source inputs
Maker-checker	Separation between the person proposing a consequential change and the approver
Modular monolith	One deployable application with explicit internal modules and boundaries